GitHub Copilot Lab

什么是 Copilot CLI Sessions?

Copilot CLI Sessions 是 VS Code 中支持在后台运行的 Agent 会话，使用 GitHub Copilot CLI 代理框架。它让你可以在 VS Code 的统一 Chat 视图中启动、监控和管理后台会话，同时你可以继续在编辑器中做其他工作。核心特点包括：

- **后台自主运行**：Copilot CLI 会话在本地机器后台独立运行，即使关闭 VS Code 窗口也不会停止
- **并行多任务**：你可以同时运行多个 Copilot CLI 会话，并行处理独立任务
- **隔离模式**：支持 Worktree（工作树）和 Workspace（工作区）两种隔离模式，保护你的代码不受干扰
- **无缝交接**：可以将本地 Agent 会话（包括完整对话历史和上下文）交接给 Copilot CLI 继续执行
- **终端集成**：支持直接在 VS Code 终端中使用 Copilot CLI

Copilot CLI Sessions 的适用场景

1. **明确范围的功能实现**：根据已有计划实施代码变更
2. **概念验证**：创建多个 POC 变体时并行工作
3. **定义清晰的修复**：执行已明确的 bug 修复或功能开发
4. **耗时任务**：需要长时间运行且不需频繁交互的任务

在本 Lab 中的应用

在本实验中，我们将：

- 了解 Copilot CLI 会话的基本概念和隔离模式
- 创建和管理 Copilot CLI 后台会话
- 体验将本地 Agent 会话交接到 Copilot CLI
- 在终端中直接使用 Copilot CLI

- 使用自定义 Agent 与 Copilot CLI 结合

实验环境要求

软件要求

- **Node.js**: >= 22.0.0
- **npm**: >= 10.0.0
- **VS Code**: 最新版本
- **GitHub Copilot**: 已登陆
- **Copilot CLI**: 已安装（可以通过 `npm install -g @github/copilot-cli`）
- **Git**: 已安装（使用 Worktree 隔离模式需要 Git 仓库）

前提条件

- 已完成 Lab 01，旅游网站项目已创建并可运行
- 当前工作区是一个 Git 仓库（使用 Worktree 隔离模式时需要）

Lab 步骤

第一步：了解 Copilot CLI 隔离模式

1.1 目标

理解 Copilot CLI 的两种隔离模式及其区别

1.2 知识要点

Copilot CLI 支持两种隔离模式来管理 Agent 对代码库的变更：

隔离模式	说明	适用场景
Worktree（工作树）	在独立文件夹中创建 Git Worktree，所有变更在 worktree 中进行，不影响主工作区	需要保护当前工作不受干扰时
Workspace（工作区）	Agent 直接在当前工作区中操作，变更就地应用	希望变更直接反映在当前工作区时

> 💡 **提示**：使用 Worktree 隔离模式时，Agent 会在每个对话轮次结束时自动提交变更到 worktree，使会话历史与提交历史保持同步。

1.3 验证

- 理解两种隔离模式的区别和各自适用场景

第二步：创建 Copilot CLI 会话

2.1 目标

学习如何在 VS Code 中创建一个 Copilot CLI 后台会话

2.2 操作步骤

1. **打开旅游网站项目**

   在 VS Code 中打开 Lab 01 创建的旅游网站项目

2. **创建新的 Copilot CLI 会话**

   你可以通过以下任一方式创建会话：

   - 打开 Chat 视图（`Ctrl + Cmd + I`），在 **Session Target** 下拉菜单中选择 **Copilot CLI** alt text
   - 点击 Chat 视图顶部的 **New Chat** 图标，选择 **New Copilot CLI Session**

- 从命令面板（`Shift + Cmd + P`）运行 `Chat: New Copilot CLI` 命令

3. **选择隔离模式**

   在弹出的选择中，选择 **Worktree** 隔离模式

   > 💡 **提示**：你可以右键点击会话列表中的会话，选择 **Open Worktree in New Window** 在新窗口中查看 worktree

4. **输入任务提示词**

   在 Chat 输入框中输入以下提示词：

   ```
   请为旅游网站添加一个页脚组件，包含版权信息、社交媒体链接和网站导航链接。
   使用与现有项目一致的技术栈和设计风格。
   ```

5. **观察后台执行**

   提交后，观察 Chat 视图中的会话状态指示器。Copilot CLI 会在后台自主执行任务，你可以继续在编辑器中做其他工作。

   > 💡 **提示**：当后台会话需要你的输入或权限确认时，状态指示器会给出提示。

2.3 验证

- Copilot CLI 会话已成功创建并在后台运行
- 可以在 Chat 视图中看到会话状态和进度
- Agent 生成的变更出现在 worktree 中

第三步：将本地 Agent 会话交接到 Copilot CLI

3.1 目标

体验将本地 Agent 对话（包括上下文）交接给 Copilot CLI 后台执行

3.2 操作步骤

1. **创建一个新的本地 Agent 会话**

   在 Chat 视图中，确保使用本地 Agent 模式（非 Copilot CLI），输入：

   ```
   我想为旅游网站添加一个目的地详情弹窗功能。当用户点击某个目的地卡片时，
   弹出一个模态窗口显示详细信息，包括大图、价格、描述和预订按钮。
   请先分析当前项目结构。
   ```

2. **与 Agent 交互确认需求**

   查看 Agent 的分析结果，如有需要可以继续对话调整需求：

   ```
   弹窗中请增加一个图片轮播功能，并添加平滑的打开/关闭动画效果
   ```

3. **交接到 Copilot CLI**

   当需求确认后，将会话交接给 Copilot CLI：

   - 打开 **Session Target** 下拉菜单，选择 **Copilot CLI**

   Copilot CLI 会话将自动启动，并携带完整的对话历史和上下文。

   > 💡 **提示**：如果你在使用 Plan Agent，也可以点击 **Start Implementation** 下拉菜单，选择 **Continue in Copilot CLI** 来在后台执行计划。

3.3 验证

- 本地 Agent 的对话上下文已成功传递给 Copilot CLI
- Copilot CLI 基于之前的对话内容继续执行任务
- 执行结果符合之前讨论的需求

第四步：在终端中使用 Copilot CLI

4.1 目标

学习如何在 VS Code 终端中直接使用 Copilot CLI

4.2 操作步骤

1. **打开 Copilot CLI 终端**

   你可以通过以下任一方式打开专用的 Copilot CLI 终端：

   - 点击终端面板中 **+** 按钮旁的下拉菜单，选择 **GitHub Copilot CLI**
   - 从命令面板（`Shift + Cmd + P`）运行 `Chat: New Copilot CLI Session` 命令
   - 在任意 VS Code 集成终端中输入 `copilot` 启动

   > 💡 **支持的 Shell**：macOS/Linux 支持 bash 和 zsh；Windows 支持 PowerShell 和 Command Prompt

2. **在终端中提交任务**

   在 Copilot CLI 终端中输入任务：

   ```
   为旅游网站项目创建一个 README.md 文件，包含项目介绍、技术栈、
   安装步骤和使用说明
   ```

3. **在 Chat 视图中跟踪**

   从终端启动的会话会自动出现在 Chat 视图的会话列表中，你可以在终端或 Chat 视图中跟踪进度和发送后续提示。

4. **恢复已有会话**

   要在终端中恢复之前的 Copilot CLI 会话，右键点击会话列表中的会话，选择 **Resume in Terminal**

> 💡 **提示**：VS Code 会自动处理 Copilot CLI 终端的身份认证，无需单独登录。

4.3 验证

- 能够成功打开 Copilot CLI 终端
- 终端中提交的任务能在 Chat 视图中同步显示
- 可以在终端中恢复已有的 Copilot CLI 会话

Copilot CLI Sessions 的限制

在使用 Copilot CLI Sessions 时，请注意以下限制：

- **工具访问受限**：Copilot CLI 会话无法访问所有 VS Code 内置工具，但可以在 Chat 输入中手动添加上下文
- **无扩展工具**：无法使用 VS Code 扩展提供的工具，仅限于 CLI 工具可用的模型
- **MCP 限制**：目前只能访问不需要身份认证的本地 MCP 服务器

总结

通过本实验，你已经学习了：

知识点	说明
Copilot CLI 会话	在后台自主运行的 Agent 会话，不影响当前工作
隔离模式	Worktree 隔离保护代码，Workspace 直接应用变更
会话交接	将本地 Agent 对话连同上下文转交给 Copilot CLI
终端使用	直接在终端中启动和管理 Copilot CLI
自定义 Agent	在 Copilot CLI 中使用自定义角色 Agent（实验性）
并行能力	同时运行多个后台会话处理不同任务